

Special Participation C1: Refactoring Transformer Notebook

Arnav Dalal, Sarvagya Somvanshi, Kabir Shah

1 Overview

For Special Participation C, we refactored the provided code for a Transformer teaching notebook (`q_hand_transformer.ipynb`). The goal was to transform the single notebook into a modular Python package structure where tests, utilities, and core logic are clearly separated in the file tree. Additionally, we adhered to PEP 8 style, PEP 257 docstring, and PEP 484 type hinting guidelines, along with commonly used style practices such as the NumPy Style Guide and Google’s Python Style Guide.

With this refactoring, we ensured that the code is more readable, maintains its pedagogical value, and is presented in a format that more closely follows typical software engineering practice seen in research and industry. We preserved the original “fill in the blanks” experience with TODO blocks for students while exposing them to a realistic project layout.

Our restructured codebase is available at:

<https://github.com/adalal07/cs182-special-participation-c2>

2 Updated File Tree

The new structure is as follows:

```
hand_transformer/
|-- q_hand_transformer.ipynb # Main notebook (student TODOs here)
|-- utils/
|   |-- __init__.py         # Package exports
|   |-- constants.py        # Configuration classes and constants
|   |-- utils.py            # General utility functions
|   |-- plotting.py         # Visualization functions
|   |-- training.py         # Training loop functions
|   |-- comparison.py       # Transformer comparison functions
|-- tests/
|   |-- __init__.py         # Test package exports
|   |-- test_transformer.py # All transformer tests (consolidated)
|-- README.md              # Setup instructions
|-- REFACTORING_NOTES.md   # High-level change summary
```

3 Detailed Refactoring Changes

3.1 From Notebook to Modular Package

Originally, `q_hand_transformer.ipynb` interleaved:

- Problem text, hints, and TODOs in markdown,
- Class and function definitions for `NumpyTransformer`, `PytorchTransformer`,
- Global constants and utility functions defined in cells, and
- Inline test code with assertions on expected outputs.

In our refactor:

- Utility functions moved into purpose-specific modules:
 - `utils/utils.py`: General utilities (`set_random_seed`, type aliases)
 - `utils/plotting.py`: Visualization (`rescale_and_plot`, `save_figure`, `get_current_figure`)
 - `utils/training.py`: Training (`train_loop`)
 - `utils/comparison.py`: Comparison (`compare_transformers`)
- All test logic consolidated into `tests/test_transformer.py`, with a shared `_run_equivalence_test()` helper function to eliminate code duplication
- Configuration classes (`PlotConfig`, `TrainingConfig`) and constants moved to `utils/constants.py`
- The refactored notebook focuses on imports, explanations, and student implementation rather than defining utilities itself
- The core transformer classes remain in the notebook because they contain student TODOs

3.2 Modular Design

File	Contents	Purpose
<code>q_hand_transformer.ipynb</code>	Classes with TODOs, transformer implementations	Student implementation
<code>utils/constants.py</code>	<code>PlotConfig</code> , <code>TrainingConfig</code> , test constants	Configuration
<code>utils/utils.py</code>	<code>set_random_seed</code> , type aliases	General utilities
<code>utils/plotting.py</code>	<code>rescale_and_plot</code> , <code>save_figure</code>	Visualization
<code>utils/training.py</code>	<code>train_loop</code>	Training
<code>utils/comparison.py</code>	<code>compare_transformers</code>	Comparison
<code>tests/test_transformer.py</code>	All test functions with shared helper	Validation

Table 1: Modular package structure

3.3 Core Logic: Transformer Implementation

In the original notebook, the `NumpyTransformer` class:

- Had minimal documentation
- Lacked type hints and comprehensive docstrings
- Had plotting code mixed with core logic

In the refactored `NumpyTransformer` class, we:

- Added PEP 484 type hints: `def forward(self, seq: NDArrayFloat, ...) -> NDArrayFloat`
- Wrote NumPy-style docstrings with mathematical formulation, learning objectives, and parameter descriptions
- Combined plotting helper functions (`_plot_matrices_and_projections` and `_plot_attention_and_output`) into a single `_plot_all` method
- Added plot saving functionality with `save_plot`, `plot_name`, and `run_name` parameters
- Applied consistent 4-space indentation per PEP 8

The mathematical formulation is now explicitly documented:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

3.4 Utilities and Tests

The original notebook declared utility functions inline. We centralized these in:

utils/constants.py:

- `PlotConfig`: Comprehensive configuration for matplotlib plots (figure size, colormap, fonts, save settings)
- `TrainingConfig`: Training hyperparameters (epochs, learning rate, optimizer settings)
- Test constants (`TEST_ITERATIONS`, `TEST_MIN_SEQ_LEN`, etc.)

utils/utils.py:

- `set_random_seed(seed: int = 42)`: Reproducibility with default seed
- Type aliases (`NDArrayFloat`, `TensorFloat`)

utils/plotting.py:

- `rescale_and_plot()`: Matrix visualization with automatic `[0, 1]` rescaling
- `save_figure()`: Save plots to local directory with timestamp and run organization
- `get_current_figure()`: Get current matplotlib figure

utils/training.py:

- `train_loop()`: Training loop with progress logging, accepts `transformer_class` parameter to avoid circular imports

utils/comparison.py:

- `compare_transformers()`: Visual comparison between hand-designed and learned transformers, with plot saving support

tests/test_transformer.py:

- `_run_equivalence_test()`: Shared helper function to eliminate code duplication across all test functions
- `test_transformer_equivalence()`: Validates NumPy and PyTorch implementations produce identical outputs
- `test_with_positional_encoding()`: Tests transformers with positional encodings
- `test_without_positional_encoding()`: Tests transformers without positional encodings
- `test_dimension_variations()`: Tests various dimension combinations
- `generate_grading_outputs()`: Generates outputs for grading purposes

Tests are called directly from the notebook:

```
1 from tests import test_transformer_equivalence
2 test_transformer_equivalence(NumpyTransformer, PytorchTransformer)
```

3.5 List of Improvements

3.5.1 Code Quality and Style

- Added comprehensive type hints (PEP 484)
- Added docstrings to all functions and classes
- Introduced type aliases (`NDArrayFloat`, `TensorFloat`)
- Used `from __future__ import annotations` for postponed evaluation
- Improved variable naming outside common ML conventions
- Consistent adherence to PEP 8, PEP 484, and PEP 257

3.5.2 Configuration Management

1. **PlotConfig** dataclass includes:

- Figure settings (width, height, DPI)
- Colormap settings (colormap, vmin, vmax)
- Text settings (font sizes)
- Axis settings (ticks, layout)
- Grid settings (rows, columns)

- Save settings (directory, format, DPI, auto-save, timestamps)

2. **TrainingConfig** provides customization for:

- Epochs, learning rate, log interval
- Optimizer selection (SGD/Adam)
- Momentum, weight decay
- Loss function selection

Benefits:

- Easily customizable options
- Defaults match original notebook behavior
- Users can override parameters without modifying source code

3.5.3 **Function Organization**

- Consolidated helper functions into a single setup cell
- Centralized all plotting into one unified function
- Added:
 - `set_random_seed()`: default seed = 42
 - `train_loop()`: added `seed`, `log_interval`
 - `test()`: added `seed` parameter
- All functions now accept configuration parameters with defaults

3.5.4 **Assertions and Validation**

- Added shape validation for inputs/outputs
- Added matrix dimension checks
- Validated intermediate tensor shapes
- Clear, concise error messages for debugging
- Benefits:
 - More informative debugging
 - Easier isolation of student mistakes

3.5.5 Plot Saving Functionality

- Introduced `save_figure()` for exporting plots
- Automatic directory creation (default `plots/`)
- Experiment grouping via `run_name`
- Supports PNG, PDF, SVG, DPI options, and timestamps
- Integrated with:
 - `NumpyTransformer.forward()`
 - `compare_transformers()`

Usage examples:

- `forward(seq, save_plot=True, plot_name="attention")`
- `compare_transformers(..., save_plot=True, run_name="exp1")`
- `save_figure(fig, "custom_plot", run_name="my_experiment")`

3.5.6 General Code Organization

- Clear separation of configuration, utilities, visualization, and training logic
- Logical grouping of related functions
- Consistent style across modules
- Improved maintainability and extensibility

3.5.7 Module Extraction and Test Consolidation

- Split helpers into:
 - `utils/utils.py` (general utilities)
 - `utils/plotting.py` (visualization)
 - `utils/training.py` (training)
 - `utils/comparison.py` (transformer comparison)
- Consolidated all tests into `tests/test_transformer.py`
- Added shared `_run_equivalence_test()` helper to remove duplication
- Maintained focused test cases for each scenario

3.5.8 Environment Compatibility

The original notebook assumed Google Colab execution. We maintained cross-platform compatibility:

- All imports work in both Colab and local environments
- No Colab-specific dependencies in utility modules
- README.md provides setup instructions for both Colab (no setup needed) and local execution (conda environment)

3.5.9 Reproducibility

We added explicit random seed initialization:

```
1 def set_random_seed(seed: int = 42) -> None:
2     """Set random seeds for reproducibility."""
3     random.seed(seed)
4     np.random.seed(seed)
5     torch.manual_seed(seed)
6     if torch.cuda.is_available():
7         torch.cuda.manual_seed_all(seed)
```

All functions that use randomness now accept an optional **seed** parameter with sensible defaults.

4 Summary of Changes

Category	Change	Reference
Modular Structure	Separated utils and tests into purpose-specific modules	Professional organization
Formatting	Consistent 4-space indentation	PEP 8
Naming	Maintained ML conventions (K, Q, V) while improving others	PEP 8
Type Hints	Added to all function signatures	PEP 484
Docstrings	NumPy-style with math formulas	PEP 257
Constants	Named constants (e.g., <code>RELATIVE_TOLERANCE</code>)	Google Style
Configuration	Dataclasses for plots and training	PEP 557
Assertions	Dimension-focused validation with clear error messages	Debugging best practices
Plot Saving	Functionality to save figures to local directory	Experiment tracking
Test Consolidation	Combined all tests with shared helper function	DRY principle

Table 2: Summary of refactoring changes

5 Style Guidelines and Citations

Our refactoring was guided by established style and engineering references:

- PEP 8 for naming, layout, and code organization (e.g., module-level docstrings, import ordering, line length, 4-space indentation)
- PEP 257 for docstring conventions (triple-quoted strings, imperative mood, parameter/return documentation)
- PEP 484 for type hints (e.g., `torch.Tensor`, `tuple[...]`, `Callable`)
- Google Python Style Guide for testing conventions, avoiding magic numbers, and module organization
- NumPy/numpydoc style for structured docstrings with Parameters, Returns, and Notes sections
- PyTorch documentation for reproducibility patterns and tensor shape conventions

6 Key Benefits

The refactoring provides several key benefits:

1. **Better Code Quality:** Type hints, docstrings, and consistent style make the code more maintainable and easier to understand

2. **Enhanced Customizability:** Comprehensive configuration classes allow users to customize behavior without modifying core code
3. **Better Organization:** Purpose-specific modules make it easier to locate and modify functionality
4. **Improved Debugging:** Strategic assertions and plot saving help users debug and track experiments
5. **Easier Experimentation:** Plot saving and seed management facilitate reproducible experiments
6. **Professional Standards:** Follows Python best practices seen in research and industry
7. **Reduced Duplication:** Consolidated test code with shared helper functions eliminates repetition
8. **Maintained Teaching Value:** All educational objectives preserved while improving code quality

7 LLM Usage

Per the Participation C prompt, we incorporated LLM assistance with Claude to refactor the assignment into a modular package with outlined style conventions

8 Conclusion

This refactoring successfully transformed the transformer notebook from a monolithic structure into a well-organized, modular package that follows software engineering best practices. The code is now more readable, maintainable, and professional while preserving all pedagogical value. Students are exposed to a realistic project layout that mirrors what they will encounter in research and industry, enhancing both their understanding of transformers and their software engineering skills.